

PROGRAM :

```
#include <stdio.h>

int fibo_recursive(int n){
    if(n <= 1)
        return n;
    return fibo_recursive(n-1) + fibo_recursive(n-2);
}

void fibo_iterative(int n){
    int a = 0, b = 1, c, i;
    printf("\nIterative Fibonacci Series:\n");
    for(i = 0; i < n; i++){
        printf("%d ", a);
        c = a + b;
        a = b;
        b = c;
    }
}

int main(){
    int n, i;
    printf("Enter number of terms: ");
    scanf("%d", &n);
    fibo_iterative(n);
    printf("\nRecursive Fibonacci Series:\n");
    for(i = 0; i < n; i++)
    {
        printf("%d ", fibo_recursive(i));
    }

    return 0;
}
```

OUTPUT :

```
C:\DAA\recursive(NEW).exe
Enter number of terms: 5

Iterative Fibonacci Series:
0 1 1 2 3
Recursive Fibonacci Series:
0 1 1 2 3
-----
Process exited after 2.625 seconds with return value 0
Press any key to continue . . .
```

PROGRAM :

```
#include <stdio.h>

int main() {
    int A[2][2], B[2][2], C[2][2];
    int i, j;
    printf("Enter elements of Matrix A (2x2):\n");
    for(i = 0; i < 2; i++)
        for(j = 0; j < 2; j++)
            scanf("%d", &A[i][j]);
    printf("Enter elements of Matrix B (2x2):\n");
    for(i = 0; i < 2; i++)
        for(j = 0; j < 2; j++)
            scanf("%d", &B[i][j]);
    int P1 = (A[0][0] + A[1][1]) * (B[0][0] + B[1][1]);
    int P2 = (A[1][0] + A[1][1]) * B[0][0];
    int P3 = A[0][0] * (B[0][1] - B[1][1]);
    int P4 = A[1][1] * (B[1][0] - B[0][0]);
    int P5 = (A[0][0] + A[0][1]) * B[1][1];
    int P6 = (A[1][0] - A[0][0]) * (B[0][0] + B[0][1]);
    int P7 = (A[0][1] - A[1][1]) * (B[1][0] + B[1][1]);
    C[0][0] = P1 + P4 - P5 + P7;
    C[0][1] = P3 + P5;
    C[1][0] = P2 + P4;
    C[1][1] = P1 + P3 - P2 + P6;
    printf("\nResult Matrix:\n");
    for(i = 0; i < 2; i++) {
        for(j = 0; j < 2; j++) {
            printf("%d ", C[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

OUTPUT :

```
C:\DAA\matrix.exe
Enter elements of Matrix A (2x2):
1 2
3 4
Enter elements of Matrix B (2x2):
5 6
7 8

Result Matrix:
19 22
43 50

-----
Process exited after 32.91 seconds with return value 0
Press any key to continue . . .
```

PROGRAM :

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, j, k;
```

```
    int graph[10][10], indegree[10] = {0};
```

```
    int visited[10] = {0};
```

```
    printf("Enter number of vertices: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter adjacency matrix:\n");
```

```
    for(i = 0; i < n; i++) {
```

```
        for(j = 0; j < n; j++) {
```

```
            scanf("%d", &graph[i][j]);
```

```
    for(i = 0; i < n; i++) {
```

```
        for(j = 0; j < n; j++) {
```

```
            if(graph[j][i] == 1)
```

```
                indegree[i]++;
```

```
    printf("Topological Order: ");
```

```
    for(k = 0; k < n; k++) {
```

```
        for(i = 0; i < n; i++) {
```

```
            if(indegree[i] == 0 && visited[i] == 0) {
```

```
                printf("%d ", i);
```

```
                visited[i] = 1;
```

```
                for(j = 0; j < n; j++) {
```

```
                    if(graph[i][j] == 1)
```

```
                        indegree[j]--;
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

OUTPUT :

```
C:\DAA\topological-sorting.exe
Enter number of vertices: 4
Enter adjacency matrix:
0 1 1 0
0 0 1 0
0 0 0 1
0 0 0 0
Topological Order: 0 1 2 3
-----
Process exited after 50.79 seconds with return value 0
Press any key to continue . . . █
```

PROGRAM :

```
#include <stdio.h>

#define INF 9999

int main() {
    int n, i, j, k;
    int dist[10][10];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter cost matrix (use 9999 for infinity):\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            scanf("%d", &dist[i][j]);
        }
    }
    for(k = 0; k < n; k++) {
        for(i = 0; i < n; i++) {
            for(j = 0; j < n; j++) {
                if(dist[i][j] > dist[i][k] + dist[k][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
    printf("\nShortest Distance Matrix:\n");
    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            printf("%d ", dist[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

OUTPUT:

```
C:\DAA\floyd.exe
Enter number of vertices: 4
Enter cost matrix (use 9999 for infinity):
0 5 9999 10
9999 0 3 9999
9999 9999 0 1
9999 9999 9999 0

Shortest Distance Matrix:
0 5 8 9
9999 0 3 4
9999 9999 0 1
9999 9999 9999 0

-----
Process exited after 44.31 seconds with return value 0
Press any key to continue . . .
```

PROGRAM :

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
struct Node {
    char data;
    unsigned freq;
    struct Node *left, *right;
};

struct Node* newNode(char data, unsigned freq) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->left = temp->right = NULL;
    temp->data = data;
    temp->freq = freq;
    return temp;
}

void swap(struct Node** a, struct Node** b) {
    struct Node* t = *a;
    *a = *b;
    *b = t;
}

void sort(struct Node* arr[], int n) {
    int i,j;
    for( i=0;i<n;i++) {
        for( j=i+1;j<n;j++) {
            if(arr[i]->freq > arr[j]->freq) {
                swap(&arr[i], &arr[j]);
            }
        }
    }
}

void printCodes(struct Node* root, int arr[], int top) {
    if(root->left) {
        arr[top] = 0;
        printCodes(root->left, arr, top+1);
    }
```

```

if(root->right) {
    arr[top] = 1;
    printCodes(root->right, arr, top+1);
if(!root->left && !root->right) {
    printf("%c: ", root->data);
    int i ;
    for( i=0;i<top;i++)
        printf("%d", arr[i]);
    printf("\n");
void HuffmanCodes(char data[], int freq[], int size) {
    struct Node* nodes[MAX];
    int i;
    for( i=0;i<size;i++)
        nodes[i] = newNode(data[i], freq[i]);
    int n = size;
    while(n > 1) {
        sort(nodes, n);
        struct Node* left = nodes[0];
        struct Node* right = nodes[1];
        struct Node* top = newNode('$', left->freq + right->freq);
        top->left = left;
        top->right = right;
        nodes[0] = top;
        nodes[1] = nodes[n-1];
        n--;
    }
    int arr[MAX], top = 0;
    printCodes(nodes[0], arr, top);
}

```

```
int main() {  
    char data[] = {'A','B','C','D','E','F'};  
    int freq[] = {5,9,12,13,16,45};  
    int size = sizeof(data)/sizeof(data[0]);  
    printf("Huffman Codes:\n");  
    HuffmanCodes(data, freq, size);  
    return 0;  
}
```

OUTPUT :

```
C:\DAA\huffman.exe
Huffman Codes:
F: 0
C: 100
D: 101
A: 1100
B: 1101
E: 111

-----
Process exited after 0.0474 seconds with return value 0
Press any key to continue . . .
```

PROGRAM :

```
#include <stdio.h>

#define ROW 3
#define COL 5

void printTable(float table[ROW][COL]) {
    int i, j;
    printf("\nSimplex Table:\n");
    for(i = 0; i < ROW; i++) {
        for(j = 0; j < COL; j++) {
            printf("%8.2f", table[i][j]);
        }
        printf("\n");
    }
}

int main() {
    float table[ROW][COL] = {
        {1, 2, 1, 0, 8},
        {3, 2, 0, 1, 12},
        {-3, -5, 0, 0, 0}
    };

    int i, j, pivot_row = -1, pivot_col;
    float ratio, min, pivot;
    printTable(table);
    while(1)
        pivot_col = -1;
    for(j = 0; j < COL-1; j++) {
        if(pivot_col == -1 || table[ROW-1][j] < table[ROW-1][pivot_col]) {
            pivot_col = j;
        }
    }
    if(table[ROW-1][pivot_col] >= 0)
        break;
    min = 9999;
    pivot_row = -1;
    for(i = 0; i < ROW-1; i++) {
```

```

        if(table[i][pivot_col] > 0) {
            ratio = table[i][COL-1] / table[i][pivot_col];
            if(ratio < min) {
                min = ratio;
                pivot_row = i;
            }
        }
        if(pivot_row == -1) {
            printf("Unbounded solution!\n");
            return 0;
        }
        pivot = table[pivot_row][pivot_col];
        for(j = 0; j < COL; j++)
            table[pivot_row][j] /= pivot;
        for(i = 0; i < ROW; i++) {
            if(i != pivot_row) {
                float factor = table[i][pivot_col];
                for(j = 0; j < COL; j++) {
                    table[i][j] -= factor * table[pivot_row][j];
                }
            }
        }

        printTable(table);
    }

    printf("\nOptimal Value (Z) = %.2f\n", table[ROW-1][COL-1]);

    return 0;
}

```

OUTPUT :

```
C:\DAA\simplex(NEW).exe

Simplex Table:
  1.00   2.00   1.00   0.00   8.00
  3.00   2.00   0.00   1.00  12.00
 -3.00  -5.00   0.00   0.00   0.00

Simplex Table:
  0.50   1.00   0.50   0.00   4.00
  2.00   0.00  -1.00   1.00   4.00
 -0.50   0.00   2.50   0.00  20.00

Simplex Table:
  0.00   1.00   0.75  -0.25   3.00
  1.00   0.00  -0.50   0.50   2.00
  0.00   0.00   2.25   0.25  21.00

Optimal Value (Z) = 21.00

-----
Process exited after 0.04874 seconds with return value 0
Press any key to continue . . .
```

PROGRAM :

```
#include <stdio.h>

#define V 4

int graph[V][V] = {
    {0, 1, 1, 0},
    {1, 0, 1, 1},
    {1, 1, 0, 1},
    {0, 1, 1, 0}
};

int path[V];

int isSafe(int v, int pos) {
    if (graph[path[pos - 1]][v] == 0)
        return 0;

    for (int i = 0; i < pos; i++)
        if (path[i] == v)
            return 0;

    return 1;
}

int hamCycleUtil(int pos) {
    if (pos == V) {
        if (graph[path[pos - 1]][path[0]] == 1)
            return 1;
        else
            return 0;
    }

    for (int v = 1; v < V; v++) {
        if (isSafe(v, pos)) {
            path[pos] = v;

            if (hamCycleUtil(pos + 1) == 1)
                return 1;

            path[pos] = -1; // backtrack
        }
    }

    return 0;
}
```

```
void hamCycle() {
    for (int i = 0; i < V; i++)
        path[i] = -1;
    path[0] = 0;
    if (hamCycleUtil(1) == 0) {
        printf("No Hamiltonian Cycle\n");
        return;
    }
    printf("Hamiltonian Cycle:\n");
    for (int i = 0; i < V; i++)
        printf("%d ", path[i]);
    printf("%d\n", path[0]);
}

int main() {
    hamCycle();
    return 0;
}
```

OUTPUT:

```
C:\DAA\hamiltonian.exe
Hamiltonian Cycle:
0 1 3 2 0

-----
Process exited after 0.06505 seconds with return value 0
Press any key to continue . . .
```

PROGRAM :

```
#include <stdio.h>

#include <limits.h>

#define N 4

int cost[N][N] = {
    {9, 2, 7, 8},
    {6, 4, 3, 7},
    {5, 8, 1, 8},
    {7, 6, 9, 4}
};

int minCost = INT_MAX;

void assign(int person, int assigned[], int currentCost) {
    if (person == N) {
        if (currentCost < minCost)
            minCost = currentCost;
        return;
    }
    for (int j = 0; j < N; j++) {
        if (!assigned[j]) {
            assigned[j] = 1;
            if (currentCost + cost[person][j] < minCost)
                assign(person + 1, assigned, currentCost + cost[person][j]);
            assigned[j] = 0; // backtrack
        }
    }
}

int main() {
    int assigned[N] = {0};
    assign(0, assigned, 0);
    printf("Minimum Assignment Cost = %d\n", minCost);
    return 0;
}
```

OUTPUT:

```
C:\DAA\Branch and bound.exe
Minimum Assignment Cost = 13
-----
Process exited after 0.06603 seconds with return value 0
Press any key to continue . . .
```

9 .Implement and compare Greedy and Dynamic Programming approaches for the Knapsack problem.

AIM :

To design efficient algorithms for **optimal resource allocation** using **Greedy and Dynamic Programming approaches** and compare their results.

ALGORITHM :

Step 1: Start

Step 2: Input number of items, weights, profits, and capacity

Step 3: Calculate profit/weight ratio for each item (Greedy)

Step 4: Sort items in descending order of ratio

Step 5: Initialize total profit = 0

Step 6: For each item:

- If capacity allows → take full item
- Else → take fractional part

Step 7: Display Greedy profit

Step 8: Initialize DP table for 0/1 Knapsack

Step 9: For each item and capacity:

- If item fits → take max(include, exclude)
- Else → exclude item

Step 10: Store maximum profit in table

Step 11: Display DP profit

PROGRAM :

```
#include <stdio.h>
```

```
void greedyKnapsack(int n, float weight[], float profit[], float capacity) {
```

```
    float ratio[10], temp, total = 0;
```

```
    for(int i = 0; i < n; i++)
```

```
        ratio[i] = profit[i] / weight[i];
```

```
    for(int i = 0; i < n - 1; i++) {
```

```
        for(int j = i + 1; j < n; j++) {
```

```
            if(ratio[i] < ratio[j]) {
```

```
                temp = ratio[i]; ratio[i] = ratio[j]; ratio[j] = temp;
```

```
                temp = weight[i]; weight[i] = weight[j]; weight[j] = temp;
```

```
                temp = profit[i]; profit[i] = profit[j]; profit[j] = temp;
```

```
    for(int i = 0; i < n; i++) {
```

```
        if(capacity >= weight[i]) {
```

```
            total += profit[i];
```

```
            capacity -= weight[i];
```

```
        } else {
```

```
            total += ratio[i] * capacity;
```

```
            break;
```

```
    printf("Greedy Profit = %.2f\n", total);
```

```
int max(int a, int b) {
```

```
    return (a > b) ? a : b;
```

```
}
```

```
void dpKnapsack(int n, int weight[], int profit[], int W) {
```

```
    int K[10][10];
```

```
    for(int i = 0; i <= n; i++) {
```

```
        for(int w = 0; w <= W; w++) {
```

```
            if(i == 0 || w == 0)
```

```
                K[i][w] = 0;
```

```
            else if(weight[i-1] <= w)
```

```
                K[i][w] = max(profit[i-1] + K[i-1][w-weight[i-1]], K[i-1][w]);
```

```

        else

            K[i][w] = K[i-1][w];

        printf("DP Profit = %d\n", K[n][W]);
int main() {
    int n = 3;

    float w1[] = {10, 20, 30};
    float p1[] = {60, 100, 120};
    int w2[] = {10, 20, 30};
    int p2[] = {60, 100, 120};
    float capacity = 50;
    greedyKnapsack(n, w1, p1, capacity);
    dpKnapsack(n, w2, p2, (int)capacity);
    return 0;
}

```

OUTPUT :

```
C:\DAA\greedy.exe
Greedy Profit = 240.00
DP Profit = 120

-----
Process exited after 0.06374 seconds with return value 0
Press any key to continue . . .
```

10. Implement and compare Backtracking and Branch & Bound for the Subset Sum problem.

AIM :

To find all subsets whose sum equals a given target using **Backtracking and Branch & Bound techniques**.

ALGORITHM :

Step 1: Start

Step 2: Initialize set of elements, target sum, and empty subset

Step 3: Check if current sum equals target → print subset

Step 4: For each element in the set:

- If (current sum + element $\leq$ target)
→ include element in subset
→ recursively call function

Step 5: Check if current sum equals target → print subset

Step 6: For remaining elements (using index):

- If (current sum + element $\leq$ target)
→ include element
→ move to next index (avoid repetition)
→ recursively call function

Step 7: Repeat until all possibilities are explored

Step 8: Stop

PROGRAM :

```
#include <stdio.h>

int n = 4, target = 10;
int set[] = {2, 4, 6, 8};
int subset[10];

void backtrack(int sum, int k) {
    if(sum == target) {
        printf("Subset (Backtracking): ");
        for(int i = 0; i < k; i++)
            printf("%d ", subset[i]);
        printf("\n"); return;
    }
    for(int i = 0; i < n; i++) {
        if(sum + set[i] <= target) {
            subset[k] = set[i];
            backtrack(sum + set[i], k + 1);
        }
    }
}

void branchBound(int sum, int k, int index) {
    if(sum == target) {
        printf("Subset (B&B): ");
        for(int i = 0; i < k; i++)
            printf("%d ", subset[i]);
        printf("\n"); return;
    }
    for(int i = index; i < n; i++) {
        if(sum + set[i] <= target) {
            subset[k] = set[i];
            branchBound(sum + set[i], k + 1, i + 1);
        }
    }
}

int main() {
    printf("Using Backtracking:\n");
    backtrack(0, 0);
    printf("\nUsing Branch & Bound:\n");
    branchBound(0, 0, 0);
    return 0;
}
```

OUTPUT :

```
C:\DAA\bound(subset).exe
Using Backtracking:
Subset (Backtracking): 2 2 2 2 2
Subset (Backtracking): 2 2 2 4
Subset (Backtracking): 2 2 4 2
Subset (Backtracking): 2 2 6
Subset (Backtracking): 2 4 2 2
Subset (Backtracking): 2 4 4
Subset (Backtracking): 2 6 2
Subset (Backtracking): 2 8
Subset (Backtracking): 4 2 2 2
Subset (Backtracking): 4 2 4
Subset (Backtracking): 4 4 2
Subset (Backtracking): 4 6
Subset (Backtracking): 6 2 2
Subset (Backtracking): 6 4
Subset (Backtracking): 8 2

Using Branch & Bound:
Subset (B&B): 2 8
Subset (B&B): 4 6

-----
Process exited after 0.0696 seconds with return value 0
Press any key to continue . . .
```

11. Analyze the performance of different sorting algorithms on various input datasets.

AIM :

To analyze and compare the performance of different sorting algorithms such as Bubble Sort, Selection Sort, and Insertion Sort on various input datasets (best case, worst case, and average case).

ALGORITHM :

Step 1: Start

Step 2: Initialize input datasets (best case, worst case, average case)

Step 3: Choose a sorting algorithm (Bubble, Selection, Insertion)

Step 4: Copy the dataset into a temporary array

Step 5: Start the timer

Step 6: Apply the sorting algorithm

Step 7: Stop the timer

Step 8: Record the time taken

Step 9: Repeat Steps 3–8 for all algorithms and datasets

Step 10: Compare the performance results

Step 11: Stop

PROGRAM :

```
#include <stdio.h>
```

```
#include <time.h>
```

```
#define N 5
```

```
void bubbleSort(int a[], int n) {
```

```
    int i,j;
```

```
    for( i = 0; i < n-1; i++)
```

```
        for( j = 0; j < n-i-1; j++)
```

```
            if(a[j] > a[j+1]) {
```

```
                int t = a[j];
```

```
                a[j] = a[j+1];
```

```
                a[j+1] = t;
```

```
void selectionSort(int a[], int n) {
```

```
    int i,j;
```

```
    for( i = 0; i < n-1; i++)
```

```
        int min = i;
```

```
        for( j = i+1; j < n; j++)
```

```
            if(a[j] < a[min]) {
```

```
                min = j;
```

```
            int t = a[i];
```

```
            a[i] = a[min];
```

```
            a[min] = t;
```

```
void insertionSort(int a[], int n) {
```

```
    int i;
```

```
    for( i = 1; i < n; i++) {
```

```
        int key = a[i];
```

```
        int j = i - 1;
```

```
        while(j >= 0 && a[j] > key) {
```

```
            a[j+1] = a[j];
```

```
            j--;
```

```
        a[j+1] = key;
```

```

void copy(int a[], int b[], int n) {
    int i;
    for( i = 0; i < n; i++) {
        b[i] = a[i];
    }
}

void testSorts(int data[], int n, char name[]) {
    int temp[N];
    clock_t start, end;
    printf("\n--- %s Dataset ---\n", name);
    copy(data, temp, n);
    start = clock();
    bubbleSort(temp, n);
    end = clock();
    printf("Insertion Sort Time = %lf seconds\n", (double)(end - start) / CLOCKS_PER_SEC);
}

int main() {
    int best[N] = {1, 2, 3, 4, 5};
    int worst[N] = {5, 4, 3, 2, 1};
    int avg[N] = {5, 3, 8, 4, 2};
    testSorts(best, N, "Best Case");
    testSorts(worst, N, "Worst Case");
    testSorts(avg, N, "Average Case");
    return 0;
};

```

OUTPUT :

```
C:\DAA\sorting.exe

--- Best Case Dataset ---
Insertion Sort Time = 0.000000 seconds

--- Worst Case Dataset ---
Insertion Sort Time = 0.000000 seconds

--- Average Case Dataset ---
Insertion Sort Time = 0.000000 seconds

-----
Process exited after 0.139 seconds with return value 0
Press any key to continue . . .
```

12 .Design an efficient algorithm to optimize resource allocation in a real world scenario.

AIM :

To design an efficient algorithm to **optimize resource allocation** such that maximum profit/value is achieved under limited resources.

ALGORITHM :

STEP 1 : Start

STEP 2 : Input number of items n

STEP 3 : Input weights and values

STEP 4 : Calculate ratio = value/weight

STEP 5 : Sort items in descending order of ratio

STEP 6: Initialize capacity W

STEP 7 : For each item:

- If weight $\leq W \rightarrow$ take full item
- Else $\rightarrow$ take fraction

STEP 8 : Calculate total profit

STEP 9 : Display result

STEP 10; Stop

PROGRAM :

```
#include <stdio.h>

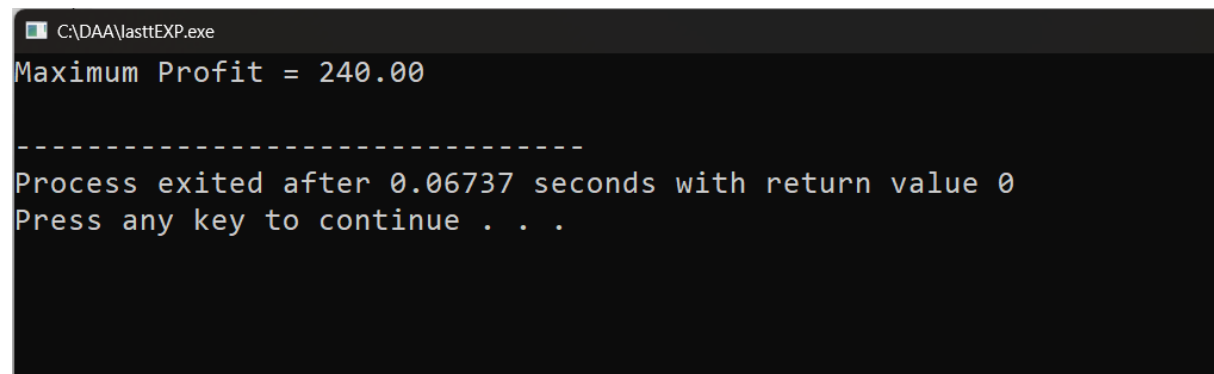
struct Item {
    int weight;
    int value;
    float ratio;
};

void sort(struct Item arr[], int n) {
    for(int i = 0; i < n-1; i++)
        for(int j = i+1; j < n; j++)
            if(arr[i].ratio < arr[j].ratio)
                struct Item temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
}

int main() {
    int n = 3;
    int capacity = 50;
    struct Item items[3] = {
        {10, 60, 0},
        {20, 100, 0},
        {30, 120, 0}
    };
    for(int i = 0; i < n; i++) {
        items[i].ratio = (float)items[i].value / items[i].weight;
    }
    sort(items, n);
    float totalProfit = 0.0;
    for(int i = 0; i < n; i++) {
        if(capacity >= items[i].weight) {
            capacity -= items[i].weight;
```

```
        totalProfit += items[i].value;
    } else {
        totalProfit += items[i].ratio * capacity;
        break;
    }
}
printf("Maximum Profit = %.2f\n", totalProfit);
return 0;
}
```

OUTPUT :



The screenshot shows a Windows command prompt window with the title bar "C:\DAA\lasttEXP.exe". The output of the program is displayed in the console:

```
Maximum Profit = 240.00
-----
Process exited after 0.06737 seconds with return value 0
Press any key to continue . . .
```

